

```
!apt-get install openjdk-11-jdk -y
```

Show hidden output

```
! pip install johnsnowlabs
```

Show hidden output

```
! pip install bertopic openai datamapplot
! pip install datasets transformers sentence-transformers
```

Show hidden output

```
#! pip install datasets
```

```
! pip install pyspark==3.5.1
```

```
Collecting pyspark==3.5.1
  Downloading pyspark-3.5.1.tar.gz (317.0 MB)
    317.0/317.0 MB 4.2 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark==3.5.1) (0.10.9.7)
Building wheels for collected packages: pyspark
  Building wheel for pyspark (setup.py) ... done
  Created wheel for pyspark: filename=pyspark-3.5.1-py2.py3-none-any.whl size=317488493 sha256=65a619085bd668e021f8dd3e8da73da39
  Stored in directory: /root/.cache/pip/wheels/b1/91/5f/283b53010a8016a4ff1c4a1edd99bbe73afac099645b5471b
Successfully built pyspark
Installing collected packages: pyspark
  Attempting uninstall: pyspark
    Found existing installation: pyspark 3.4.0
    Uninstalling pyspark-3.4.0:
      Successfully uninstalled pyspark-3.4.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the
johnsnowlabs 6.2.0 requires pyspark==3.4.0, but you have pyspark 3.5.1 which is incompatible.
Successfully installed pyspark-3.5.1
```

This code initiates a spark session through spark-nlp and displays the version of Spark NLP and Spark that is currently running in our notebook.

```
import sparknlp
spark = sparknlp.start()

print("Spark NLP version: {}".format(sparknlp.version()))
print("Apache Spark version: {}".format(spark.version))
```

```
Spark NLP version: 6.2.0
Apache Spark version: 3.5.1
```

Here you will need to upload your dataset (reviews.csv) that you downloaded from <http://insideairbnb.com/get-the-data/> and extracted. This dataset contains the actual reviews of the airbnb stays that we have referenced throughout the other homeworks this semester. Note: In this code I have connected my Google Drive but you may also just upload your file in the temp folders available in Google colab

```
reviewsDF = spark.read.options(mode='FAILFAST',
                               header= True, multiline = True, inferSchema=True, escape='')\
.csv('/content/reviews.csv.gz')
```

This code is something that I did to limit the amount of data that I was using from the dataset. This is to conserve on time sitting at the computer. The more data that you include in your model, the closer that your model will be to the actual topics but is more computationally expensive and will take a lot more time since we are running in a psuedo-spark environment and not an actual cluster computing scenario.

Make your choice about how accurate you want to be and how long you want to wait for the code to run.

```
### Split Dataset
(extraDF,useDF) = reviewsDF.randomSplit((0.60,0.40),seed=42)
```

A bit of data-engineering of assigning our text column to a variable name and cleaning out rows that are missing in the dataset.

```
from pyspark.sql import functions as F

text_col = 'comments'
review_text = useDF.select("listing_id","id",text_col).filter(F.col(text_col).isNotNull())
```

✓ Screenshot #1 -

Take a screen shot of the top 5 comments in your new dataset.

```
review_text.head(5)
```

```
[Row(listing_id=1913774, id=8865628, comments="Brian was an awesome host! The apartment was recently remodeled and definitely felt like a new home. I'd definitely stay here again. I love his place!"),
 Row(listing_id=1913774, id=9431986, comments='Brian was the absolute, best host. If we needed anything he provided it. His studio is very sweet too. The nine windows on three walls makes it feel very spacious. It is decorated very tastefully. Highly recommended!'),
 Row(listing_id=1913774, id=9915528, comments="great place! great view! easy to walk to everything! great, friendly, easy-to-communicate-with host! totally awesome dogs! i hope to be moving to bozeman, but if not and i'm just visiting i'd definitely want to stay there again. thanks, brian, for making it so easy!"),
 Row(listing_id=1913774, id=10386565, comments="My fiance and I had an absolutely amazing time staying at Brian's place in Bozeman! The place is very clean, modern, and extremely comfortable! We arrived to a bottle of wine, beer and water in the fridge, and warm coffee and tea any time we wanted it. Brian was very friendly, welcoming, laid back, and helpful! I cannot say enough good things about our stay here. I think the number one way you can truly show how much you enjoyed a place is by using the same place to stay when you return to that city...and that is exactly what we'll be doing anytime we come to Bozeman! We had plenty of privacy, and if we needed anything at all Brian was only a text or phone call away. I HIGHLY RECOMMEND staying at this beautiful loft right next to downtown Bozeman."),
 Row(listing_id=1913774, id=10819586, comments='Brian was a a fantastic host. He was helpful with providing advice for places to go around town. The room was well equipped and exceptionally well put together. Would definitely stay here again.')]

```

```
review_text.show(5, truncate=False)
```

```
+-----+-----+-----+
|listing_id|id      |comments
+-----+-----+-----+
|1913774   |8865628 |Brian was an awesome host! The apartment was recently remodeled and definitely felt like a new home. I'd de
|1913774   |9431986 |Brian was the absolute, best host. If we needed anything he provided it. His studio is very sweet too. Th
|1913774   |9915528 |great place! great view! easy to walk to everything! great, friendly, easy-to-communicate-with host! totall
|1913774   |10386565|My fiance and I had an absolutely amazing time staying at Brian's place in Bozeman! The place is very clea
|1913774   |10819586|Brian was a a fantastic host. He was helpful with providing advice for places to go around town. The room w
+-----+-----+-----+
only showing top 5 rows
```

✓ BERTopic Modeling with HuggingFace (<https://github.com/MaartenGr/BERTopic>)

```
from bertopic import BERTopic
```

```
topic_model = BERTopic( embedding_model=embedding_model, umap_model=umap_model, hdbscan_model=hdbscan_model,
verbose=True ).fit(text, embeddings)
```

BERTopic works rather straightforward. It consists of 5 sequential steps:

-

 1. Embedding documents
 2. Reducing the dimensionality of embeddings
 3. Cluster reduced embeddings
 4. Tokenize documents per cluster
 5. Extract best-representing words per cluster

https://colab.research.google.com/drive/1bxizKzv5vfxJEB29sntU_ZC7PBSIPaQ?usp=sharing#scrollTo=cadvhCBTc6Zz

Convert Dataset to HuggingFace with Datasets

✓ Convert each split to Hugging Face Dataset

Topic models could even be guided to see how many documents contain known potential topics in the corpus of documents...

https://maartengr.github.io/BERTopic/getting_started/guided/guided.html

```
from datasets import *

reviews = Dataset.from_spark(review_text)
#from_spark is converting the pyspark dataframe to a hugging dataframe
```

✓ Embeddings (BERTTopics uses Sentence Transformers)

(https://www.sbert.net/docs/sentence_transformer/pretrained_models.html)

Running embeddings before model. Typically, we want to iterate fast over different versions of our BERTopic model whilst we are trying to optimize it to a specific use case. To speed up this process, we can pre-compute the embeddings, save them, and pass them to BERTopic so it does not need to calculate the embeddings each time:

```
from sentence_transformers import SentenceTransformer

# Create an embedding for each abstract
embedding_model = SentenceTransformer('sentence-transformers/distilbert-base-nli-mean-tokens')
embeddings = embedding_model.encode(reviews['comments'], show_progress_bar=True)
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
modules.json: 100% 229/229 [00:00<00:00, 19.9kB/s]
config_sentence_transformers.json: 100% 122/122 [00:00<00:00, 13.5kB/s]
README.md: 3.80k/? [00:00<00:00, 331kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 6.31kB/s]
config.json: 100% 550/550 [00:00<00:00, 63.5kB/s]
model.safetensors: 100% 265M/265M [00:02<00:00, 154MB/s]
tokenizer_config.json: 100% 450/450 [00:00<00:00, 53.6kB/s]
vocab.txt: 232k/? [00:00<00:00, 7.32MB/s]
tokenizer.json: 466k/? [00:00<00:00, 27.8MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 11.6kB/s]
config.json: 100% 190/190 [00:00<00:00, 19.3kB/s]
Batches: 100% 587/587 [00:26<00:00, 86.57it/s]
```

✓ BERTTopics Model Pipeline Build

✓ UMAP (<https://github.com/lmcinnes/umap>)

```
from umap import UMAP
```

```
umap_model = UMAP(n_neighbors=15, n_components=5, min_dist=0.0, metric='cosine', random_state=42)
```

UMAP is one of a few dimensionality reduction algorithms that keeps a significant portion of the high-dimensional local structure in lower dimensionality.

here are a number of parameters that can be set for the UMAP class; the major ones are as follows:

- **n_neighbors**: This determines the number of neighboring points used in local approximations of manifold structure. Larger values will result in more global structure being preserved at the loss of detailed local structure. In general this parameter should often be in the range 5 to 50, with a choice of 10 to 15 being a sensible default.
- **min_dist**: This controls how tightly the embedding is allowed compress points together. Larger values ensure embedded points are more evenly distributed, while smaller values allow the algorithm to optimise more accurately with regard to local structure. Sensible values are in the range 0.001 to 0.5, with 0.1 being a reasonable default.

- `metric`: This determines the choice of metric used to measure distance in the input space. A wide variety of metrics are already coded, and a user defined function can be passed as long as it has been JITd by numba.

You can play around with the values of dimensionality to optimize for your topic creation. Note that too low dimensionality results in a loss of information while a too high dimensionality results in poorer clustering results.

```
from umap import UMAP
umap_model = UMAP(n_neighbors=12,
                  n_components=5,
                  min_dist=0.01,
                  metric='cosine',
                  random_state=42)
```

✓ HDBSCAN (<https://github.com/scikit-learn-contrib/hdbSCAN>)

```
from hdbSCAN import HDBSCAN
```

```
hdbSCAN_model = HDBSCAN(min_cluster_size=20, min_samples=2, metric='euclidean', cluster_selection_method='eom')
```

Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN). Performs DBSCAN over varying epsilon values and integrates the result to find a clustering that gives the best stability over epsilon. Used in transformer topic modeling after dimension reduction.

```
from hdbSCAN import HDBSCAN
hdbSCAN_model = HDBSCAN(min_cluster_size=20,
                        min_samples=2,
                        metric='euclidean',
                        cluster_selection_method='eom')
```

```
/usr/local/lib/python3.12/dist-packages/hdbSCAN/robust_single_linkage_.py:175: SyntaxWarning: invalid escape sequence '\{'
$max \{ core_k(a), core_k(b), 1/\alpha d(a,b) \}$.
```

✓ Vectorizers

CountVectorizer

Converts a collection of text documents to a matrix of token counts. You also have the ability to remove common words in english that do not carry much meaning ('stop words'). BERTopics works here with a sklearn package tool.

```
from sklearn.feature_extraction.text import CountVectorizer
```

c-TF-IDF

In order to get an accurate representation of the topics from our bag-of-words matrix, TF-IDF was adjusted to work on a cluster/categorical/topic level instead of a document level. This adjusted TF-IDF representation is called c-TF-IDF and takes into account what makes the documents in one cluster different from documents in another cluster. This can also be used a type of filter in the construction of topics saying that we want to reduce the number of similar words in the topics.

```
from bertopic import BERTopic from bertopic.vectorizers import ClassTfidfTransformer
```

```
ctfidf_model = ClassTfidfTransformer() topic_model = BERTopic(ctfidf_model=ctfidf_model )
```

There are two main options (which can be used together) -

- `bm25_weighting` is a boolean parameter that indicates whether a class-based BM-25 weighting measure is used instead of the default method as defined in the formula at the beginning of this page.
- `reduce_frequent_words` to take the square root of the term frequency after applying the weighting scheme.

```
from sklearn.feature_extraction.text import CountVectorizer
from bertopic.vectorizers import ClassTfidfTransformer

vectorizer = CountVectorizer(stop_words="english")

#ctfidf
ctfidf_model = ClassTfidfTransformer(bm25_weighting=True, reduce_frequent_words=True)
```

```
from bertopic import BERTopic
from bertopic.representation import MaximalMarginalRelevance
```

```
representation_model = MaximalMarginalRelevance(diversity=0.2)
topic_model = BERTopic(representation_model=representation_model)
```

`nr_topics=` This command lets you change the number of topics and 'auto' lets the system decide on the number of topics though this does not always produce the best results.

CHANGE THE NUMBER OF TOPICS from the number in this code and run the model a couple of times to see how your topics change with the different number of topics.

You can change the number of topics as you need to depending on the dataset and there are several methods and philosophies on doing this. This is one of many packages that will evaluate topics (<https://github.com/mind-Lab/octis>)

```
# Train our topic model with BERTopic
from bertopic import BERTopic

topic_model = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer,
    nr_topics= 15, # you could set this to 'auto' and it will assign the topics automatically
    ctfidf_model=ctfidf_model,
    representation_model=representation_model
).fit(reviews['comments'], embeddings)
```

✓ **FULL MODEL IN ONE CELL** (*Note: you do not need to run this cell if you have run the previous cells building the model*)

```
# Train our topic model with BERTopic limiting the number of topics
from bertopic import BERTopic
from umap import UMAP
from hdbscan import HDBSCAN
from sklearn.feature_extraction.text import CountVectorizer
from bertopic.vectorizers import ClassTfidfTransformer

# Define sub-models
vectorizer = CountVectorizer(stop_words="english")
umap_model = UMAP(n_neighbors=10, min_dist=0.01, metric='cosine', random_state=42)
hdbscan_model = HDBSCAN(min_cluster_size=20, min_samples=2, metric='euclidean', cluster_selection_method='eom')
ctfidf_model = ClassTfidfTransformer(bm25_weighting=True, reduce_frequent_words=True)

topic_model = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer,
    nr_topics=28,
    ctfidf_model=ctfidf_model
).fit(reviews['comments'], embeddings)
```

✓ Screen Shot #2

Model Output

```
topic_model.get_topic(##)
```

This command will allow us to see the topics that our run of the model has created. The output shows:

Topic number Numbering starts at -1 but this topic can mostly be ignored if it is not containing all of your count because of how the BERTopics model uses this -1 topic to store outliers in it's model.

Count - count of topic occurrence in the dataset

****Name** ** model given name using the top three or four words in the topic

Representation - top ten words contributing to the topic

Representative_Docs - a single document (a review in our use case) that is representative of the topic.

====

Run the `topic_model.get_topic()` call and take a screenshot of the output produced.

topic_model.get_topic_info()

	Topic	Count	Name	Representation	Representative_Docs
0	-1	5249	-1_enjoyed_loved_stayed_cozy	[enjoyed, loved, stayed, cozy, needed, town, t...	[Lark was very flexible and friendly. She was...
1	0	5768	0_coffee_kitchen_airbnb_loved	[coffee, kitchen, airbnb, loved, walk, family,...	[This was our very first Airbnb reservation. ...
2	1	1299	1_helpful_responsive_host_communication	[helpful, responsive, host, communication, acc...	[Such an amazing place to stay! Super clean an...
3	2	1218	2_distance_walkable_walking_quiet	[distance, walkable, walking, quiet, restauran...	[This place is perfect for your Bozeman stay! ...
4	3	891	3_communication_convenient_exactly_good	[communication, convenient, exactly, good, exc...	[Beautiful spot! Comfortable, clean and cute.,...
5	4	722	4_montana_yellowstone_ski_state	[montana, yellowstone, ski, state, skiing, sky...	[What a wonderful Airbnb!! Super cute and ...
6	5	511	5_basement_bedroom_noise_shower	[basement, bedroom, noise, shower, bathroom, u...	[Exceptionally clean with great amenities! Com...
7	6	389	6_heart_weekend_base_visiting	[heart, weekend, base, visiting, spot, getaway...	[I would absolutely stay in this little gem ag...
8	7	331	7_enjoyed_good_vey_thanks	[enjoyed, good, vey, thanks, thank, wonderful,...	[Nice place, good location, had a really good ...
9	8	312	8_brilliant_spot_good_awesome	[brilliant, spot, good, awesome, advertised, t...	[great spot, as advertised!, Great spot. As a...
10	9	283	9_uncomfortable_did_sleep_loud	[uncomfortable, did, sleep, loud, night, dirty...	[worst host experience ever. 0 stars . worst ...
11	10	268	10_second_definitely_repeat_book	[second, definitely, repeat, book, come, forwa...	[A repeat stay for us and the place was just a...
12	11	235	11_beds_bed_comfy_shower	[beds, bed, comfy, shower, sleep, bathroom, pi...	[This spot is cute and cozy - great linens, co...
13	12	197	12_hosts_host_accommodating_awesome	[hosts, host, accommodating, awesome, staff, a...	[Everything was great and exactly as described...
14	13	153	13_house_beautiful_home_comforts	[house, beautiful, home, comforts, yard, apart...	[Beautiful home. Very comfortable and a wonder...
15	14	123	14_dog_dogs_joe_yard	[dog, dogs, joe, yard, fenced, huck, hiliary, p...	[Great spot, the fenced backyard was great for...
16	15	121	15_kitchen_meals_cooking_cook	[kitchen, meals, cooking, cook, stocked, equip...	[The house is awesome! Great location - conven...
17	16	110	16_near_close_right_main	[near, close, right, main, stadium, downtown, ...	[Great place right near the heart of downtown!...

一切都很完美 设施非常全 虽然早 房间是地下室 价格比较实惠 加里临时

Top Ten Words

This is the top 10 words of your BERTopics model topics. Take a screenshot of the words and see if you can assign names for the different topics as you understand them based on the words in your printout. If you would like to see more words from each topic then change the 'num_top_words' variable.

topic_model.topic_representations_

Show hidden output

Screen Shot #3 -

Another way to look at the top ten words in a dataframe and not a python dictionary. Take a screenshot of your topics and move the scroll at the bottom if needed to get all topics in the screenshot.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import lit
import pandas as pd

topic_representations = topic_model.topic_representations_

max_terms = max(len(words) for words in topic_representations.values())

data_dict = {}
for topic_id, word_scores in topic_representations.items():
    words = [f"{word} ({score:.4f})" for word, score in word_scores]
    while len(words) < max_terms:
        words.append(None)
    data_dict[f"Topic_{topic_id}"] = words

pandas_df = pd.DataFrame(data_dict)
spark_df = spark.createDataFrame(pandas_df)

# Show result
spark_df.show(truncate=False)
```

Topic_-1	Topic_0	Topic_1	Topic_2	Topic_3	Topic_4
enjoyed (0.1476)	coffee (0.1609)	helpful (0.2478)	distance (0.2786)	communication (0.2787)	montana (0.3622)
loved (0.1449)	kitchen (0.1515)	responsive (0.2322)	walkable (0.2626)	convenient (0.2684)	yellowstone (0.3420)
stayed (0.1438)	airbnb (0.1493)	host (0.2303)	walking (0.2598)	exactly (0.2570)	ski (0.2673)
cozy (0.1434)	loved (0.1479)	communication (0.2279)	quiet (0.2548)	good (0.2562)	state (0.2556)
needed (0.1429)	walk (0.1473)	accommodating (0.2271)	restaurants (0.2488)	excellent (0.2510)	skiing (0.2428)
town (0.1425)	family (0.1470)	hosts (0.1992)	neighborhood (0.2452)	advertised (0.2498)	sky (0.2300)
time (0.1424)	stocked (0.1469)	respond (0.1968)	downtown (0.2358)	value (0.2487)	mountains (0.2299)
family (0.1423)	neighborhood (0.1454)	described (0.1953)	walk (0.2336)	cute (0.2481)	mountain (0.2281)
staying (0.1423)	needed (0.1452)	friendly (0.1946)	close (0.2315)	thank (0.2448)	bridger (0.2267)
just (0.1422)	staying (0.1447)	super (0.1930)	street (0.2258)	spot (0.2434)	snow (0.2227)

Topic Labels

The generate_topic_labels command will automatically put the top words as a label. This does not really describe the topics in a human sense. That is your job.

```
topic_model.generate_topic_labels(nr_words=4)
```

```
['-1_enjoyed_loved_stayed_cozy',
 '0_coffee_kitchen_airbnb_loved',
 '1_helpful_responsive_host_communication',
 '2_distance_walkable_walking_quiet',
 '3_communication_convenient_exactly_good',
 '4_montana_yellowstone_ski_state',
 '5_basement_bedroom_noise_shower',
 '6_heart_weekend_base_visiting',
 '7_enjoyed_good_vey_thanks',
 '8_brilliant_spot_good_awesome',
 '9_uncomfortable_did_sleep_loud',
 '10_second_definitely_repeat_book',
 '11_beds_bed_comfy_shower',
 '12_hosts_host_accommodating_awesome',
 '13_house_beautiful_home_comforts',
 '14_dog_dogs_joe_yard',
 '15_kitchen_meals_cooking_cook',
 '16_near_close_right_main',
 '17_一切都很完美_设施非常全_虽然是地下室_生活配套设施齐全',
 '18_quiet_comunucation_whit_accuracy',
 '19_second_2nd_time_year',
 '20_hostel_soon_rent_come',
 '21_ve_airbnb_best_favorite',
 '22_muy_la_el_es',
 '23_na_gg_aa_hostile',
 '24_couldn_asked_bro_staycationing',
 '25_longer_wish_stayed_trendiest',
 '26_ski_skiing_bridger_bowl']
```

Another variation of looking at the top 10 words in the topics and the weight assigned to that word to represent the topic.

```
topic_model.get_topic(3)

[('communication', np.float64(0.2787061829903962)),
 ('convenient', np.float64(0.26837510961219785)),
 ('exactly', np.float64(0.25700264177364884)),
 ('good', np.float64(0.25620835927231356)),
 ('excellent', np.float64(0.25101833876499047)),
 ('advertised', np.float64(0.24977387924715636)),
 ('value', np.float64(0.2487214157594743)),
 ('cute', np.float64(0.24812305253855943)),
 ('thank', np.float64(0.24478489279896393)),
 ('spot', np.float64(0.24343656298454236))]
```

Command lets us see the frequency of each topic identified by the model.

[Show hidden output](#)[Show hidden output](#)[Show hidden output](#)

Now we can add another application to our understanding of the dataset and see if the topics being discussed can be used to predict whether or not the host is a superhost. We will use the `RandomForestClassifier` from the included `pyspark ml` library.

```
listingsDF = spark.read.options(mode='FAILFAST',
                                header = True,
                                multiline=True,
                                inferSchema=True,
                                escape='\"').csv("/content/listings (2).csv") #You will need to replace this with your file path
```

```
listingsDF = listingsDF.select("id","host name","host id","number of reviews", "price")
```

```
import pandas as pd
df = pd.DataFrame({"comments": reviews['comments'], "Topic": topic_model.topics_, "listing_id": reviews['listing_id'] })
documentTopics df = spark.createDataFrame(df)
```

```
pivot df = documentTopics df.groupBy("listing id").pivot("Topic").agg(F.count("Topic"))
```

```
result df = pivot df.groupBy("listing id").agg(*[F.sum(c).alias(c) for c in pivot df.columns if c != "listing id"]).fillna(0)
```

```
result df.show()
```

listing_id	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	
50675728	8	22	4	3	2	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0

744478255073405873	12	29	7	1	2	7	2	0	2	1	0	0	1	2	2	0	1	0	1	0	0	1	1	0	0	0	0
9740033	11	5	0	0	4	0	2	0	0	0	3	1	0	0	3	0	0	0	0	0	0	1	2	0	1	1	0
45075319	47	59	17	8	12	7	2	5	2	4	2	7	4	3	1	0	0	0	0	1	0	1	1	0	0	0	1
1108037590849320235	2	2	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1040565103561819864	10	14	5	8	2	0	3	0	3	0	0	0	0	1	0	0	0	1	0	0	0	0	2	0	1	0	
1314082323149340257	3	0	2	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
25580821	8	9	2	3	5	4	0	0	0	1	1	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	
46570803	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
684167798771337575	13	14	4	5	2	1	2	0	0	0	0	1	1	0	0	0	0	1	0	0	0	0	0	0	1	0	
22031083	2	3	0	1	0	2	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	1	
705082802450406622	8	5	1	3	4	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	
53281258	2	5	0	1	0	3	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1113152204237576730	1	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
14769819	20	26	2	0	4	4	5	0	1	1	3	1	1	3	0	0	0	0	0	1	0	1	0	0	0	0	
51825573	23	15	4	3	4	1	1	1	0	0	1	0	0	0	2	0	0	1	0	0	1	0	0	0	0	0	
607365320794077990	1	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
39002591	38	51	8	6	6	9	1	5	5	1	2	0	3	0	0	0	2	1	0	0	0	1	0	0	1	0	
1950545	6	4	1	1	1	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	
577635693880635570	13	14	3	4	4	2	1	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	0	0	

only showing top 20 rows

```
totalsDF = result_df.join(listingsDF,result_df.listing_id == listingsDF.id).drop(listingsDF.id)
totalsDF.show(truncate=True)
```

listing_id	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
50675728	8	22	4	3	2	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0
744478255073405873	12	29	7	1	2	7	2	0	2	1	0	0	1	2	2	0	1	0	1	0	0	1	1	0	0	0	0
9740033	11	5	0	0	4	0	2	0	0	3	1	0	0	3	0	0	0	0	0	0	1	2	0	1	1	0	
45075319	47	59	17	8	12	7	2	5	2	4	2	7	4	3	1	0	0	0	0	1	0	1	1	0	0	0	1
1108037590849320235	2	2	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1040565103561819864	10	14	5	8	2	0	3	0	3	0	0	0	0	1	0	0	0	1	0	0	0	0	2	0	1	0	
1314082323149340257	3	0	2	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
25580821	8	9	2	3	5	4	0	0	0	1	1	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	
46570803	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
684167798771337575	13	14	4	5	2	1	2	0	0	0	0	1	1	0	0	0	0	1	0	0	0	0	0	0	1	0	
22031083	2	3	0	1	0	2	0	0	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	1	
705082802450406622	8	5	1	3	4	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	
53281258	2	5	0	1	0	3	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1113152204237576730	1	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
14769819	20	26	2	0	4	4	5	0	1	1	3	1	1	3	0	0	0	0	0	1	0	1	0	0	0	0	
51825573	23	15	4	3	4	1	1	1	0	0	1	0	0	0	2	0	0	1	0	0	1	0	0	0	0	0	
607365320794077990	1	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
39002591	38	51	8	6	6	9	1	5	5	1	2	0	3	0	0	0	2	1	0	0	0	1	0	0	1	0	
1950545	6	4	1	1	1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	
577635693880635570	13	14	3	4	4	2	1	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	0	0	

only showing top 20 rows

✓ Screenshot #4 -

Here I am converting the 'host_is_superhost' column into a 'dummy variable' where it will be a binary (1,0) column simplified to 'superhost'. Take a screenshot of the show() call of the dataframe before going into the transformation for the RandomForest model.

Note: Again be mindful that transformations will take some computation time.

```
from pyspark.ml.feature import StringIndexer
totalsDF = StringIndexer(
    inputCol='host_id',
    outputCol='superhost',
    handleInvalid='keep').fit(totalsDF).transform(totalsDF)

totalsDF.show(5, truncate=True)
```

listing_id	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
50675728	8	22	4	3	2	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0
744478255073405873	12	29	7	1	2	7	2	0	2	1	0	0	1	2	2	0	1	0	1	0	0	1	1	0	0	0	0
9740033	11	5	0	0	4	0	2	0	0	3	1	0	0	3	0	0	0	0	0	0	1	2	0	1	1	0	
45075319	47	59	17	8	12	7	2	5	2	4	2	7	4	3	1	0	0	0	0	1	0	1	1	0	0	0	1
1108037590849320235	2	2	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

only showing top 5 rows

Here we are defining the independent variables that we are going to include into our RandomForestClassifier model. You can decide to include additional variables from the listings.csv dataframe and build out a robust model but here I am only including the discovered topics from the text of the reviews.

Note: This will relate to the number of topics that you have run in your model. Do not run this code blindly without adjusting the numbers to match your model

```
# Code to print out column names. Copy and paste the topics to make the coding of required features faster
column_names = totalsDF.columns
print(column_names)
```

```
['listing_id', '-1', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16', '17', '18', '19']
```

Builds up a ML pipeline through feature engineering and then establishes a RandomForest Classifier model

```
required_features = ['-1', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14'] # Make sure that you

from pyspark.ml.feature import VectorAssembler
assembler = VectorAssembler(inputCols=required_features, outputCol='features')
transformed_data = assembler.transform(totalsDF)
```

```
(training_data, test_data) = transformed_data.randomSplit([0.8,0.2])
```

```
from pyspark.ml.classification import RandomForestClassifier
rf = RandomForestClassifier(labelCol='superhost',
                           featuresCol='features',
                           maxDepth=5)
```

ON a large dataset be prepared for this to take awhile to run or to time out in our Google Colab notebook

```
model = rf.fit(training_data)
```

```
predictions = model.transform(test_data)
```

```
# Evaluate our model
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
evaluator = MulticlassClassificationEvaluator(
    labelCol='superhost',
    predictionCol='prediction',
    metricName='accuracy')
```

✓ Screenshot #5 -

This is the printout of the accuracy of the model predicting whether or not the airbnb host is a superhost. Take a screenshot of the printout cell of your models test accuracy. Though this is not the end all, be all for evaluating a RandomForest Classifier, it is a helpful measure in seeing how useful your model is making a prediction.

A next step is to look at the importance of features in the model when assisting the classification model. This can be an important component of our interpretation if we can look at different factors that really influence the predictions that we are making.

```
accuracy = evaluator.evaluate(predictions)
print('Test Accuracy = ', accuracy)

import pandas as pd
featureImp = pd.DataFrame(
    list(zip(assembler.getInputCols(), model.featureImportances)), columns=["features", "importance"])
featureImp.sort_values(by="importance", ascending=False)
```

```
Test Accuracy = 0.06451612903225806
```

	features	importance
1	0	0.151979
0	-1	0.144171
2	1	0.068934
10	9	0.068827
4	3	0.065086
7	6	0.060477
6	5	0.059197
5	4	0.059173
14	13	0.058920
3	2	0.056358
12	11	0.048261
9	8	0.042793
11	10	0.036070
8	7	0.027809
13	12	0.026480
15	14	0.025465

```
topic_model.get_topic_info()
```

Command allows us to see the main components of our topic modeling on the dataset and provides a representative document for each topic.

```
topic_model.get_topic_info()
```

	Topic	Count	Name	Representation	Representative_Docs
0	-1	5249	-1_enjoyed_loved_stayed_cozy	[enjoyed, loved, stayed, cozy, needed, town, t...	[Lark was very flexible and friendly. She was...
1	0	5768	0_coffee_kitchen_airbnb_loved	[coffee, kitchen, airbnb, loved, walk, family,...	[This was our very first Airbnb reservation. ...
2	1	1299	1_helpful_responsive_host_communication	[helpful, responsive, host, communication, acc...	[Such an amazing place to stay! Super clean an...
3	2	1218	2_distance_walkable_walking_quiet	[distance, walkable, walking, quiet, restauran...	[This place is perfect for your Bozeman stay! ...
4	3	891	3_communication_convenient_exactly_good	[communication, convenient, exactly, good, exc...	[Beautiful spot! Comfortable, clean and cute,...
5	4	722	4_montana_yellowstone_ski_state	[montana, yellowstone, ski, state, skiing, sky...	[What a wonderful Airbnb!! Super cute and ...
6	5	511	5_basement_bedroom_noise_shower	[basement, bedroom, noise, shower, bathroom, u...	[Exceptionally clean with great amenities! Com...
7	6	389	6_heart_weekend_base_visiting	[heart, weekend, base, visiting, spot, getaway...	[I would absolutely stay in this little gem ag...
8	7	331	7_enjoyed_good_vey_thanks	[enjoyed, good, vey, thanks, thank, wonderful,...	[Nice place, good location, had a really good ...
9	8	312	8_brilliant_spot_good_awesome	[brilliant, spot, good, awesome, advertised, t...	[great spot, as advertised!, Great spot. As a...
10	9	283	9_uncomfortable_did_sleep_loud	[uncomfortable, did, sleep, loud, night, dirty...	[worst host experience ever. 0 stars . worst ...
11	10	268	10_second_definitely_repeat_book	[second, definitely, repeat, book, come, forwa...	[A repeat stay for us and the place was just a...
12	11	235	11_beds_bed_comfy_shower	[beds, bed, comfy, shower, sleep, bathroom, pi...	[This spot is cute and cozy - great linens, co...
13	12	197	12_hosts_host_accommodating_awesome	[hosts, host, accommodating, awesome, staff, a...	[Everything was great and exactly as described...
14	13	153	13_house_beautiful_home_comforts	[house, beautiful, home, comforts, yard, apart...	[Beautiful home. Very comfortable and a wonder...
15	14	123	14_dog_dogs_joe_yard	[dog, dogs, joe, yard, fenced, huck, hiliary, p...	[Great spot, the fenced backyard was great for...
16	15	121	15_kitchen_meals_cooking_cook	[kitchen, meals, cooking, cook, stocked, equip...	[The house is awesome! Great location - conven...
17	16	110	16_near_close_right_main	[near, close, right, main, stadium, downtown, ...	[Great place right near the heart of downtown!...
			17_一切都很完美 设施非常全 虽然是地下室 牛	[一切都很完美, 设施非常全, 虽然是	[房间是地下室, 价格比较实惠, 如果临时

✓ Screenshot #6

BONUS - Go through some of the John Snow Labs documentation and tutorials (<https://nlp.johnsnowlabs.com/models?type=pipeline>) and https://github.com/JohnSnowLabs/spark-nlp-workshop/tree/master/tutorials/Certification_Trainings/Public

And apply a new technique to our dataset. You will have to write your own code here. You do not need a new dataset (not another city download) but a new analytics applied to the same dataset that you already have preloaded.

```
# SCREENSHOT 6 - Basic text classification with MLlib

from pyspark.ml.feature import Tokenizer, HashingTF, IDF
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

# Creating a dummy sentiment label (1 for long reviews, 0 for short)
reviews_labeled = review_text.withColumn(
    "label",
    (F.length(F.col("comments")) > 100).cast("integer")
)

# Tokenize and transform text
tokenizer = Tokenizer(inputCol="comments", outputCol="words")
hashingTF = HashingTF(inputCol="words", outputCol="rawFeatures", numFeatures=1000)
idf = IDF(inputCol="rawFeatures", outputCol="features")
```

```
lr = LogisticRegression(maxIter=10)

pipeline = Pipeline(stages=[tokenizer, hashingTF, idf, lr])

model = pipeline.fit(reviews_labeled)
```